

B15 Series B

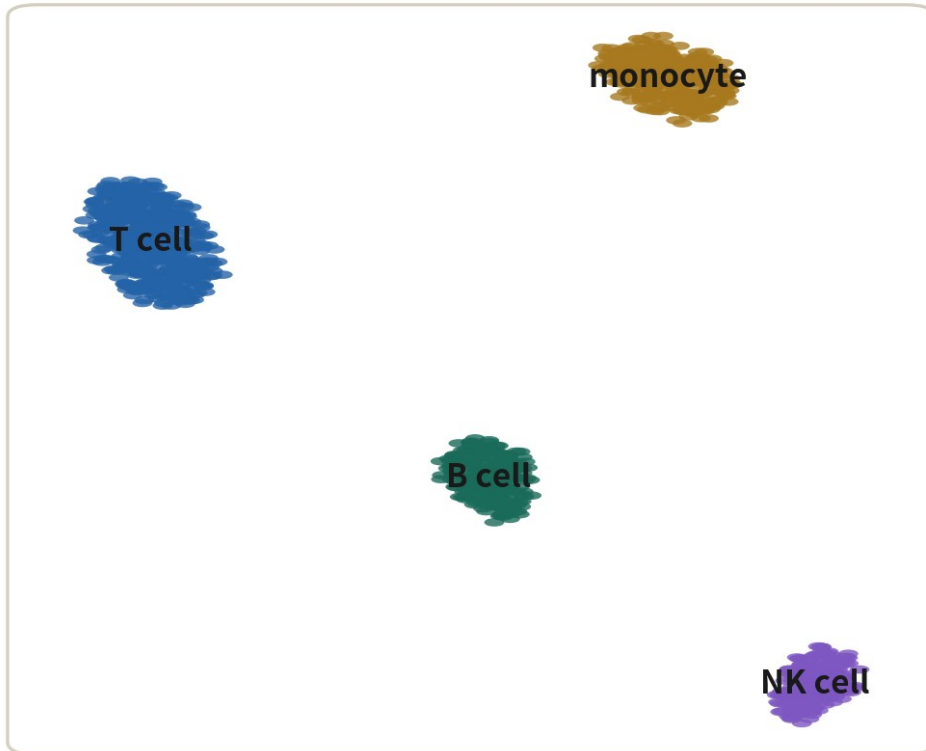
Trajectory analysis and pseudotime: a time axis for cell states

Decide whether your data deserves a trajectory — then build a pseudotime with a root, a direction, and answers.

About 40 min | Prerequisites: B7-B9 | Data: simulated differentiation + PBMC counterexample

Force-feeding PBMC to a trajectory tool

your PBMC: four discrete types

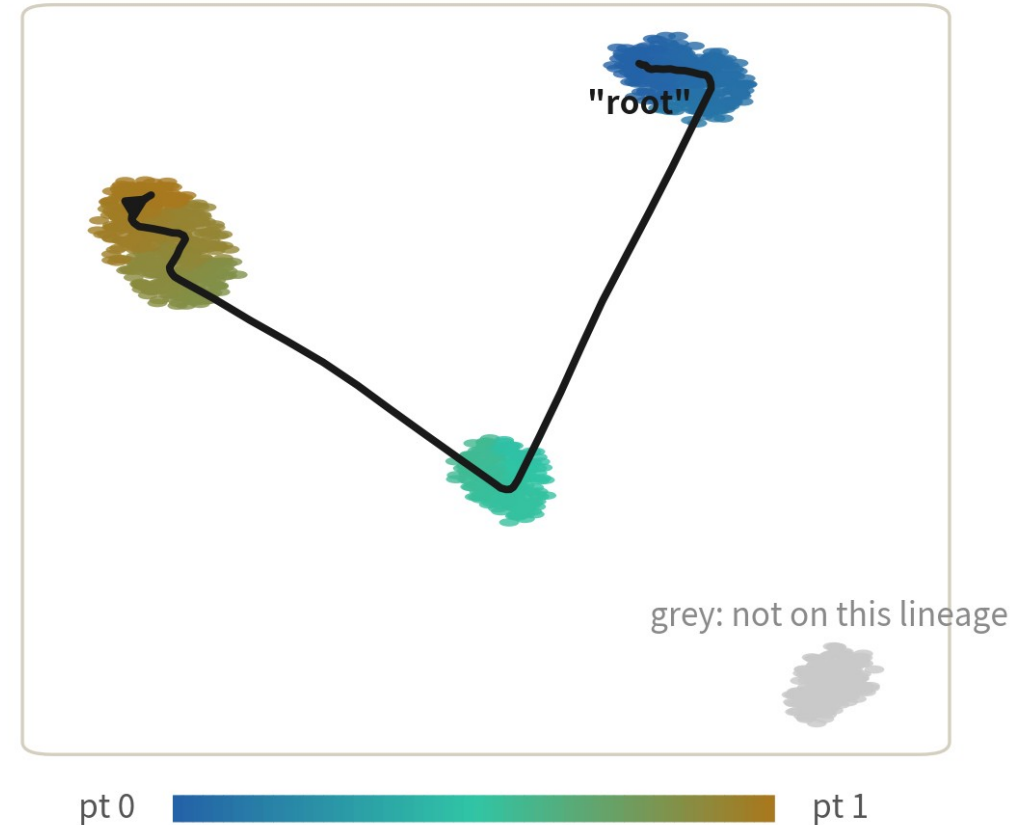


**no cell is differentiating between these mature types
— yet the curve gets drawn**

simulated data (algorithms run for real)

the tool still hands you a "differentiation path"

same
dataset
← →



Left: four mature, discrete types. Right: the tool still draws a lovely 'differentiation path', pseudotime included

WHERE THIS EPISODE STARTS

**The tool will always hand you a curve.
The question is whether that curve exists.**

Trajectory tools throw no 'not applicable' error. Judging suitability is your job, not theirs.

What you will learn

- 1 Judge with a three-question checklist whether a dataset deserves a trajectory — and why PBMC fails it
- 2 Run Slingshot for real: clusters → MST → principal curves → pseudotime, and see how the root decides everything
- 3 Read gene dynamics along the path the tradeSeq way, and place Monocle3, PAGA and RNA velocity on the map

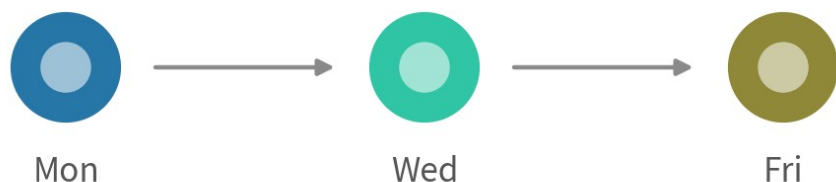
01

The snapshot paradox

You never had a time axis — only one photo taken at one instant

How one snapshot becomes a timeline

the fantasy: filming one cell over time



sequencing kills the cell: one photo per cell, ever

the reality: one snapshot at one instant



order them by similarity → a pseudo time axis

pseudotime is an ordering, not a clock

schematic

Sequencing kills the cell, so tracking is impossible — but one snapshot catches cells at different progress, and similarity can order them

Three questions before you run anything

1

1. is the process continuous

states change gradually — not jumps
between fixed types

bone marrow: yes / mature PBMC: no

2

2. are intermediates sampled

cells must actually populate
the road between endpoints

a continuous bridge: yes / islands: no

3

3. is the root biologically known

the root comes from prior knowledge:
stemness markers, time points

the algorithm never picks it for you

fail any one of the three and hold off — the tool will not stop you from running anyway

- Continuous? States must grade into each other, not jump between types
- Intermediates sampled? The road between endpoints must be populated
- Root known? It comes from prior biology, never from the algorithm
- Fail any one question — hold off

Which space to compute in

PCA space: compute the trajectory here

MST & principal curves

distances mean something here

pseudotime = arc length on the curve

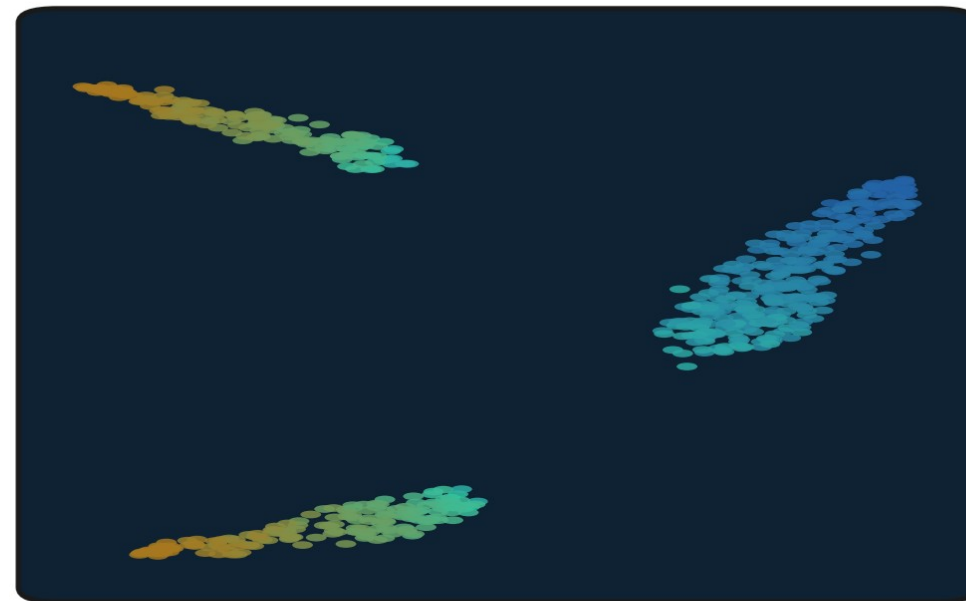
it literally measures a path length

slingshot(..., reducedDim = "PCA")

the default in this episode

compute,
then project

UMAP: for your eyes only



you may draw the curve on it, but never compute pseudotime here —
B9: UMAP distances cannot be trusted

simulated data (algorithms run for real)

Trajectories and pseudotime live in PCA space; UMAP only displays. The B9 rule stands: UMAP distances cannot be trusted

CORE IDEA

**A trajectory is a hypothesis,
not a discovery.**

You assumed continuity and direction; the tool merely draws your assumptions. Wrong assumptions still draw beautifully.

02

Build the demo data

One trunk, two branches — and we hold the answer key

Simulate a differentiation process

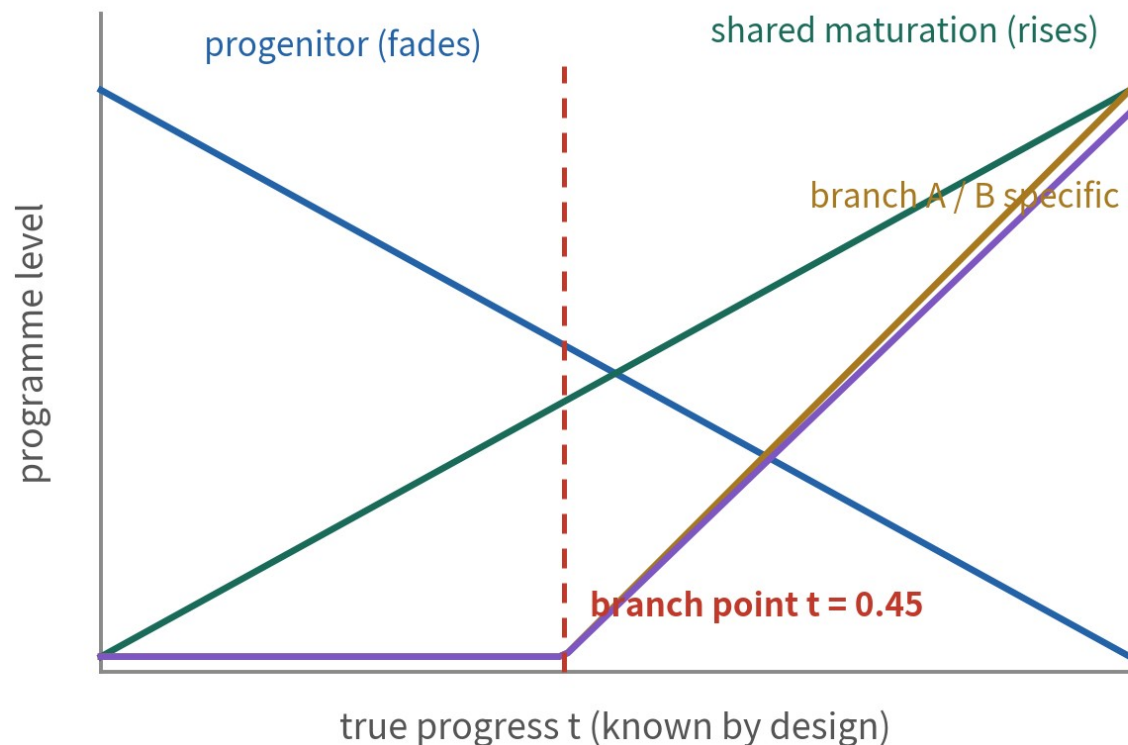
```
set.seed(1234)          # 全系列固定
n <- 1500; tb <- 0.45    # 細胞數; 分支點在 t = 0.45
t.true <- sort(runif(n)) # 每顆細胞的真實分化進度
branch <- ifelse(t.true < tb, 0, # 過了分支點, 隨機走 A 或 B
                 sample(1:2, n, replace = TRUE))
after <- pmax(t.true - tb, 0) / (1 - tb)

# 四組基因程式 (splatter 的 path 概念, 自寫版):
# 起點程式衰減、共同成熟上升、A / B 特異分支點後啟動
mu <- base + outer(1 - t.true, p.start) + outer(t.true, p.mature) +
      outer(after * (branch == 1), p.brA) +
      outer(after * (branch == 2), p.brB)
counts <- matrix(mbinom(length(mu), mu = t(mu), size = 2),
                 nrow = 2000) # 負二項取樣: 2,000 基因 × 1,500 顆
```

The point of simulating: t.true is known, so we can grade the algorithm later

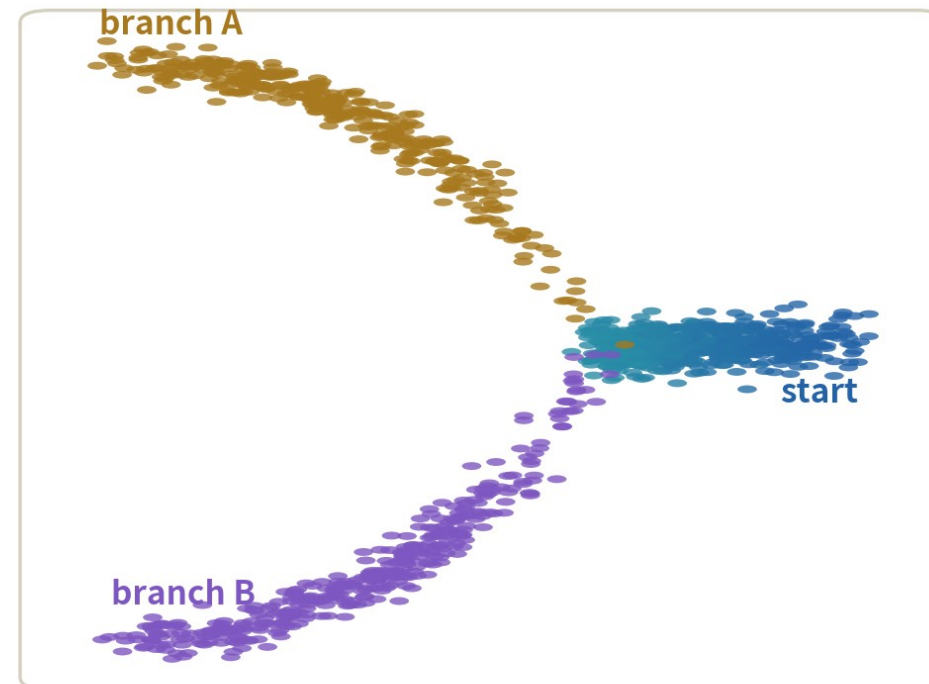
What the simulation looks like

the gene programmes, by design



simulated data (algorithms run for real)

PC1-PC2: one trunk, two branches



1,300 cells $\times$ 900 genes, negative binomial sampling

Left: the four designed gene programmes. Right: trunk and branches emerge on PC1-PC2

Standard preprocessing, through clustering

```
sim <- CreateSeuratObject(counts = counts, project = "sim ")
sim <- NormalizeData(sim) |> FindVariableFeatures() |>
  ScaleData() |> RunPCA(npcs = 20) |>
  RunUMAP(dims = 1:10)
sim <- FindNeighbors(sim, dims = 1:10) |>
  FindClusters(resolution = 0.4)
table(sim$seurat_clusters)
```

0	1	2	3	4	5
412	330	289	198	152	119

The whole B5-B9 pipeline, untouched — trajectory analysis comes after clustering

What you have at this point

A differentiation dataset



2,000 genes $\times$ 1,500
cells: trunk plus two
branches

The answer key



each cell's true
progress t.true and
branch identity

Six clusters



exactly what Slingshot
eats: labels + PCA
coordinates

03

Slingshot, for real

getLineages' MST → getCurves → pseudotime

getLineages: the skeleton first

```
library(slingshot)
sce <- as.SingleCellExperiment(sim)
rd <- reducedDim(sce, "PCA")[, 1:2] # 在 PCA 空間算

lin <- getLineages(rd, sim$seurat_clusters,
                  start.clus = "5") # root 是「你」指定的
slingLineages(lin)
```

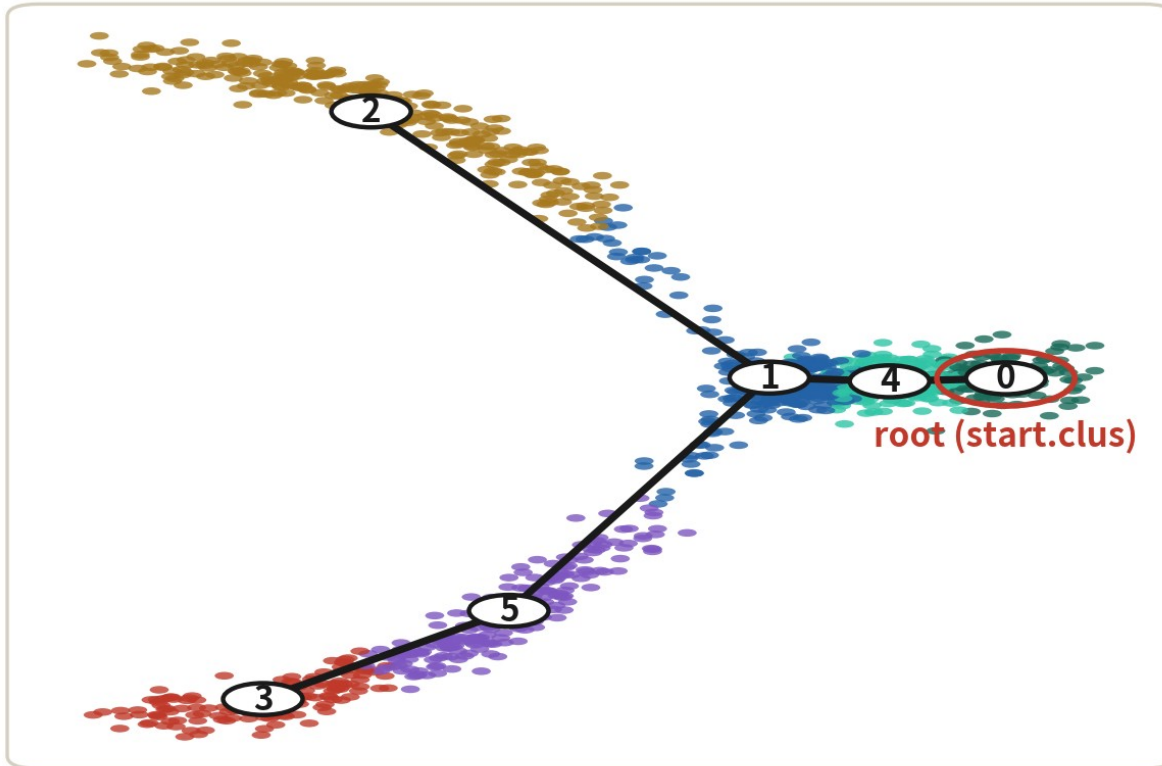
```
$Lineage1
[1] "5" "0" "1" "3" "2"

$Lineage2
[1] "5" "0" "1" "4"
```

There is no default to copy for start.clus — that is the point: the root is a biological decision

The MST: clusters strung into a tree

clusters + centroids + MST



simulated data (algorithms run for real)

The minimum spanning tree links nearest clusters; every fork adds a lineage

what getLineages returns

Lineage1:

0 → 4 → 1 → 5 → 3

Lineage2:

0 → 4 → 1 → 2

one lineage per leaf of the tree

the MST links nearest clusters;
branches come from forks in the tree

getCurves: polyline to smooth curve

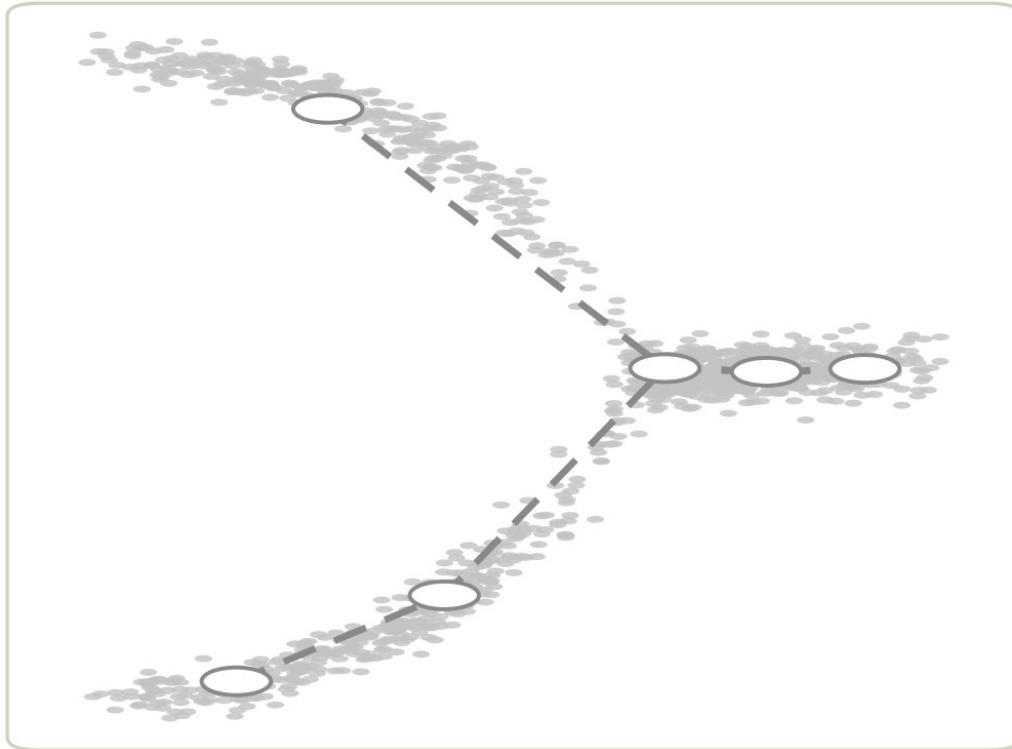
```
crv <- getCurves(lin)      # principalcurve 迭代拟合  
crv
```

```
class: Pseudotime ordering  
pathStats(2): pseudotime weights  
cellnames(1500): cell_1 cell_2 ... cell_1500  
pathnames(2): Lineage1 Lineage2
```

One principal curve per lineage: project, smooth, re-project, until convergence

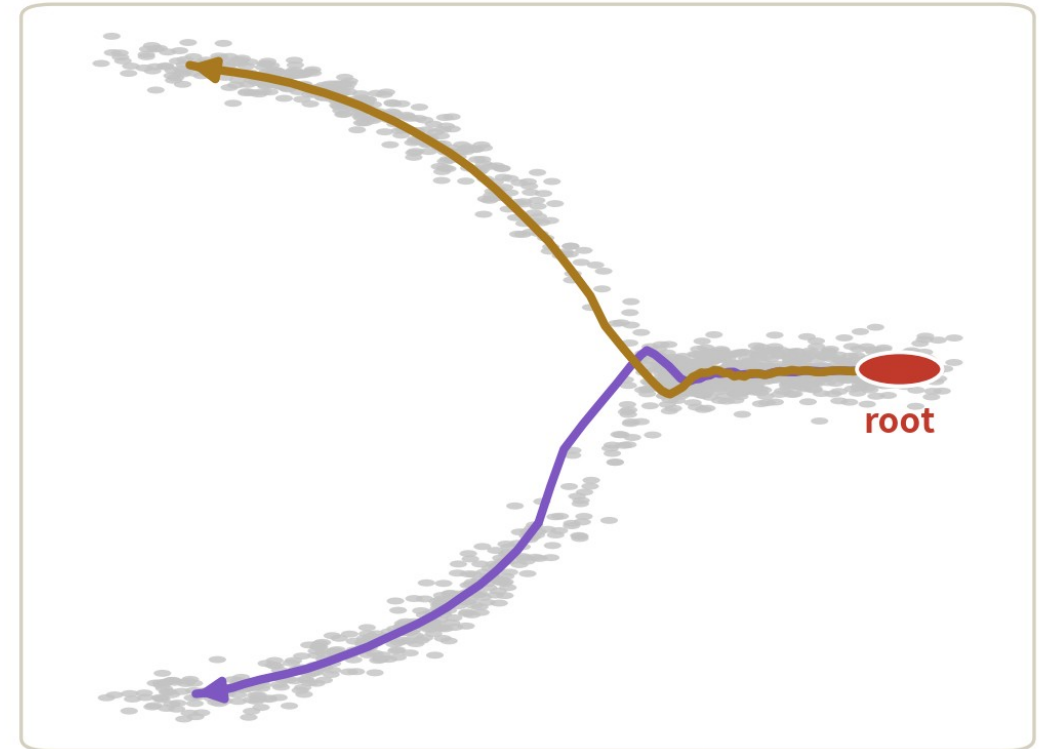
What the principal curves look like

the MST polyline: cluster-level only



simulated data (algorithms run for real)

getCurves: smooth principal curves



iterate: project $\rightarrow$ local averages $\rightarrow$ re-project, until converged

The polyline stops at cluster resolution; the curve paves the road down to cell resolution

Extract the pseudotime

```
pt <- slingPseudotime(crv) # 細胞 × lineage 矩陣  
head(pt, 3)
```

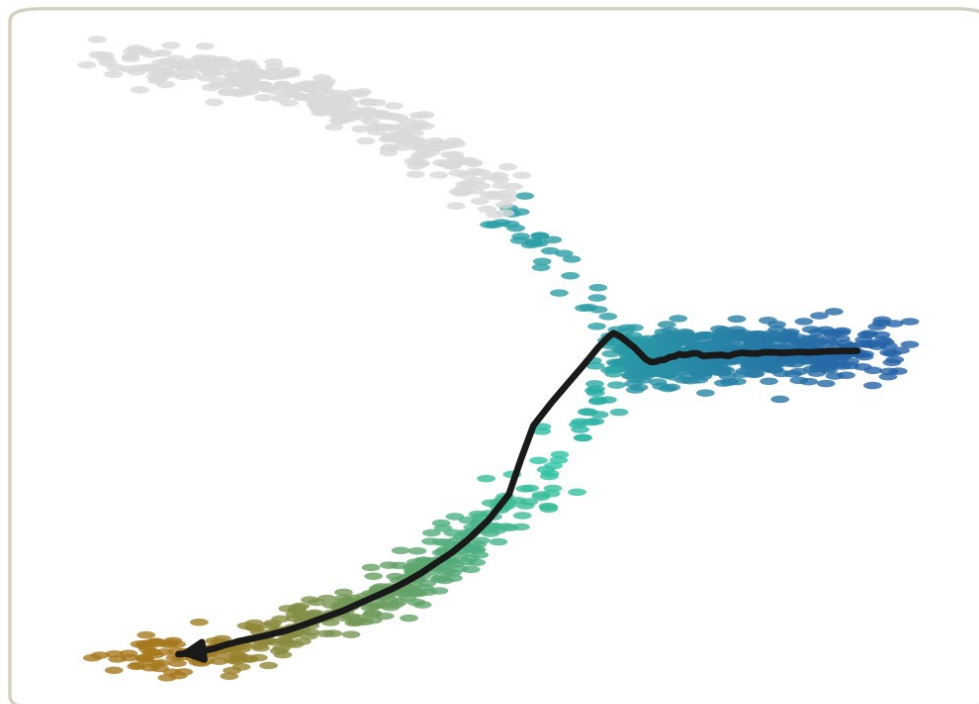
```
sim$pt1 <- pt[colnames(sim), "Lineage1"]  
FeaturePlot(sim, "pt1", reduction = "umap") +  
  scale_colour_viridis_c(na.value = "grey85")
```

	Lineage1	Lineage2
cell_1	0.31	0.29
cell_2	2.87	NA
cell_3	1.04	1.12

NA is not an error — that cell sits on the other branch, off this lineage

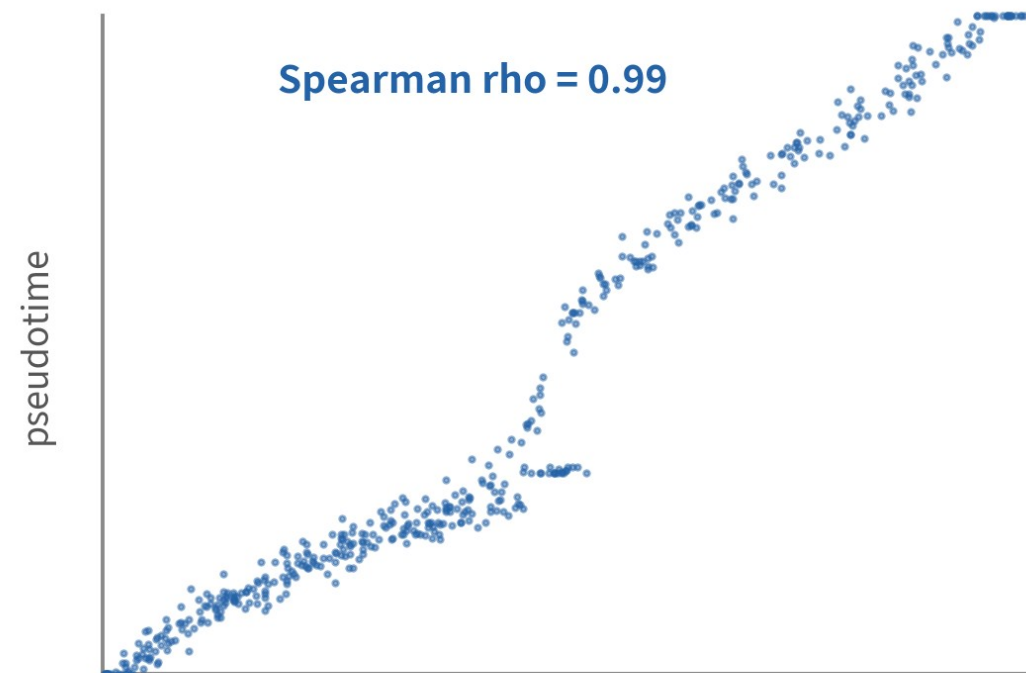
Grading pseudotime against the truth

pseudotime along Lineage1



early  late
simulated data (algorithms run for real)

checked against the simulated truth



true progress t (known by design)
the ordering is right — but the values are neither hours nor days

Spearman rho = 0.99 against true progress: the ordering is right. The values are still neither hours nor days

The three Slingshot steps, in one card each

getLineages



centroids + MST
decide which clusters
the path visits, and
where it forks

getCurves



principal curves sand
the branches into
smooth roads

slingPseudotime



each cell's arc length
along its curve is its
pseudotime

04

The root experiment

Swap the root, rerun, and watch the direction flip

Same data, root moved to a branch tip

```
## [踩雷示範] root 亂選：拿分支末端的群當起點
lin2 <- getLineages(rd, sim$seurat_clusters,
  start.clus = "2") # 唯一的改動
crv2 <- getCurves(lin2)
pt2 <- slingPseudotime(crv2)

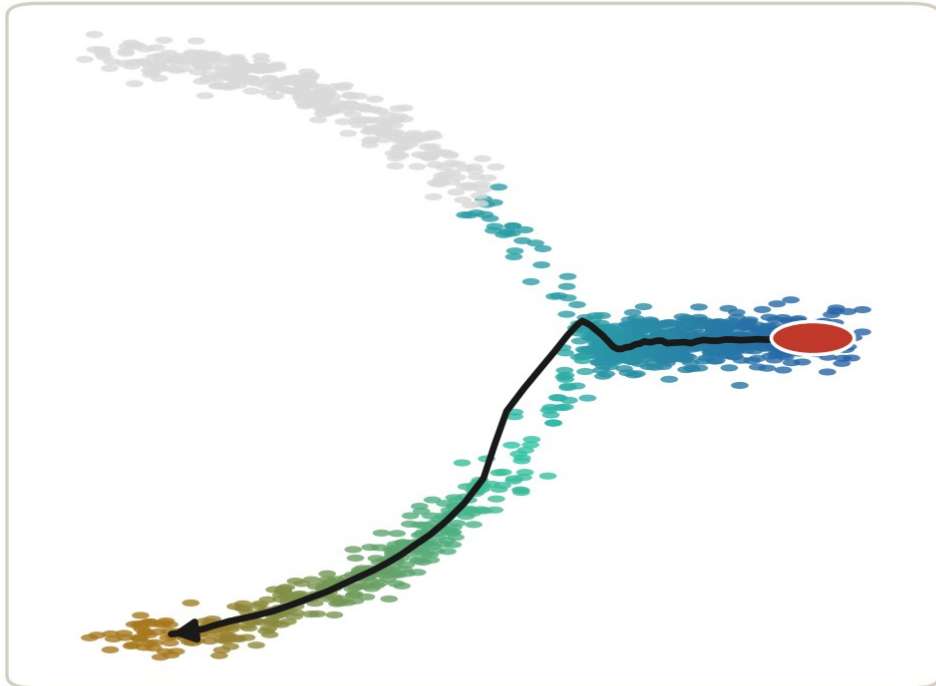
cor(pt[, "Lineage1"], pt2[, "Lineage1"],
  method = "spearm an", use = "complete.obs")
```

```
[1] -0.998
```

A correlation of -1: the same cells, ordered exactly backwards

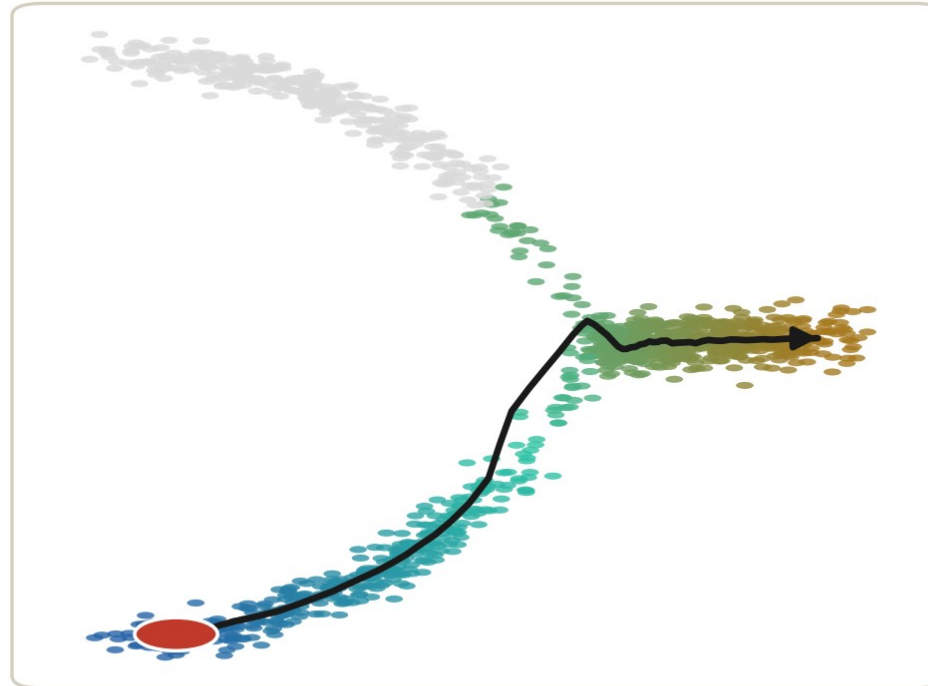
The reversal, with your own eyes

root = progenitor cluster (justified)



early  late
simulated data (algorithms run for real)

root = tip of branch A (arbitrary)



swap the root and the start of differentiation
becomes its end — the data never changed

the diff

curve:
identical

direction:
reversed

$\rho = -1$

Identical curve, opposite pseudotime — the start of differentiation becomes its end

FOR YOUR METHODS SECTION

**"The root was set at cluster X,
based on stemness marker Y."**

You must be able to state the evidence: markers, time points, or a known hierarchy. If you cannot, revisit checklist question three.

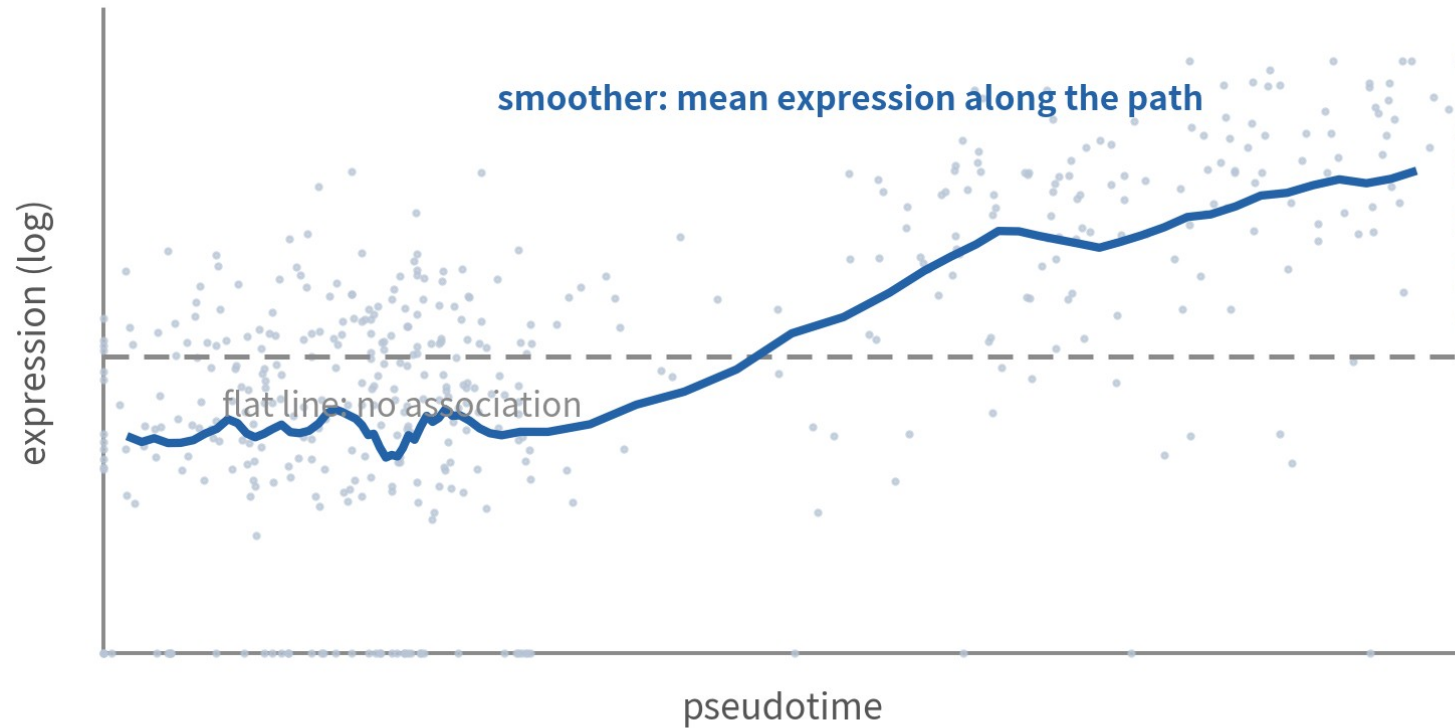
05

Gene dynamics along the path

The tradeSeq way: fitGAM → associationTest → heatmap

The smoother: from haze to curve

one branch B-specific gene along Lineage1



simulated data (algorithms run for real)

Single-cell expression is haze; the smooth curve along pseudotime is the readable signal

what tradeSeq does

fitGAM

fit one smooth curve of expression vs pseudotime per gene

associationTest

test: is the curve flat?
if not, expression tracks the path

between lineages

diffEndTest etc.: how endpoints of two branches differ

tradeSeq: fit and test

```
library(tradeSeq)
# 每個基因擬合「表達 ~ pseudotime」的 NB-GAM 平滑曲線
gam <- fitGAM(counts = counts(sce), sds = crv,
              nknots = 6)      # 幾千基因要跑數分鐘

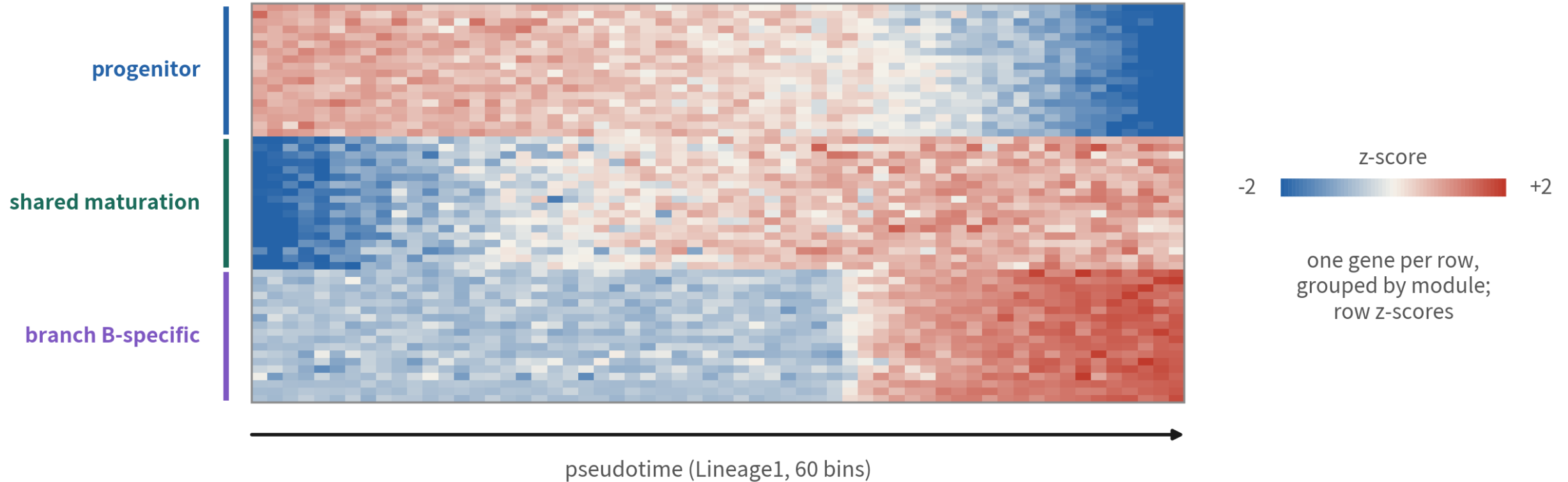
assoc <- associationTest(gam)  # 檢定：曲線是水平線嗎
head(assoc[order(assoc$pvalue), ], 3)
```

	waldStat	df	pvalue	meanLogFC
G0261	212.4	5	3.1e-44	2.61
G0088	198.7	5	2.9e-41	2.35
G0293	185.2	5	6.8e-39	2.20

associationTest finds genes that track the path; branch differences go to diffEndTest / patternTest

Expression heatmap along the path

54 genes × pseudotime: three modules take turns



simulated data (algorithms run for real)

Cells ordered by pseudotime, genes by module: progenitor fades, maturation lights up, the branch programme takes over last

Three disciplines for reading that heatmap

Read smoothers, not dots



raw per-cell values are
too noisy; conclusions
live on the curves

Test along the curve



use `associationTest` —
never split pseudotime
in half for a t-test

Modules need biology



each module must
map to a biological
programme, or it is
only pretty colours

06

The ecosystem map

Beyond Slingshot: when to upgrade, and to what

Four tools, four pieces of the puzzle

Slingshot

clusters + MST + principal curves.
simple, stable, interpretable;
first choice for tree-like paths

this episode

Monocle3

learns a principal graph on UMAP.
handles loops & disconnects;
at the cost of embedding artefacts

complex topology

PAGA

cluster-level connectivity (scanpy).
asks which clusters connect
before any path is drawn

scout first

RNA velocity

spliced/unspliced ratios give
each cell a velocity vector;
direction from splicing, not a root

for direction

tradeSeq sits outside this row: it takes pseudotime from any method and owns the downstream DE

decide whether to run at all first — then start from the simplest tool

Trees: Slingshot. Complex topology: Monocle3. Scouting: PAGA. Evidence for direction: RNA velocity

07

Where people get hurt

Three traps — each demonstrated for real

Trap 1: forcing a trajectory through discrete types

What it is

Feeding mature PBMC types to a trajectory tool and reporting the output curve as a differentiation path.



Why it happens

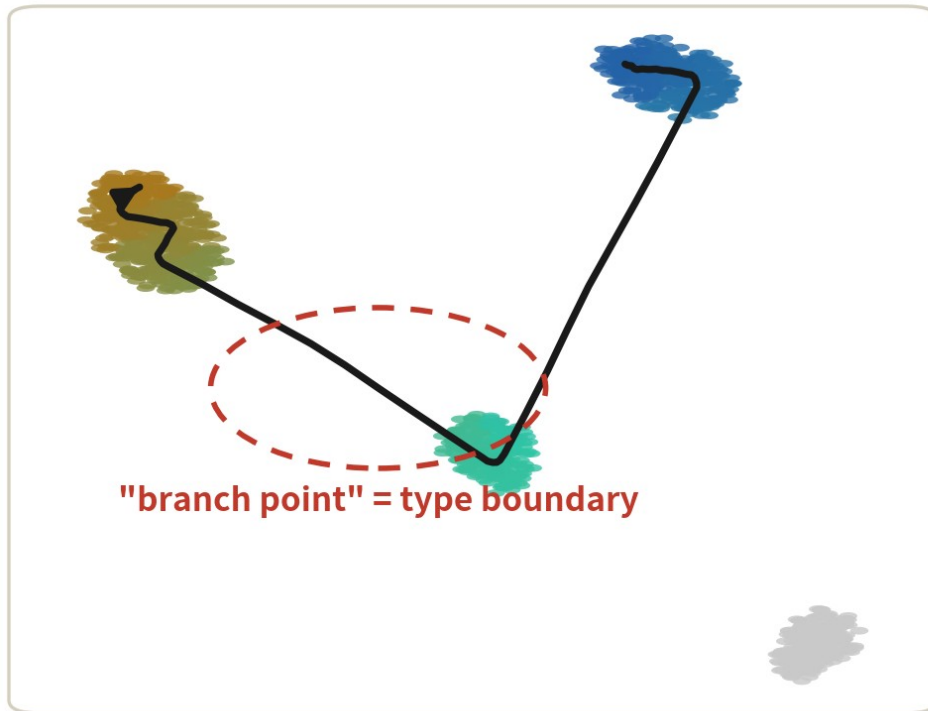
The tool never checks the premise and always returns a curve — and curves naturally look like stories.

What it costs you

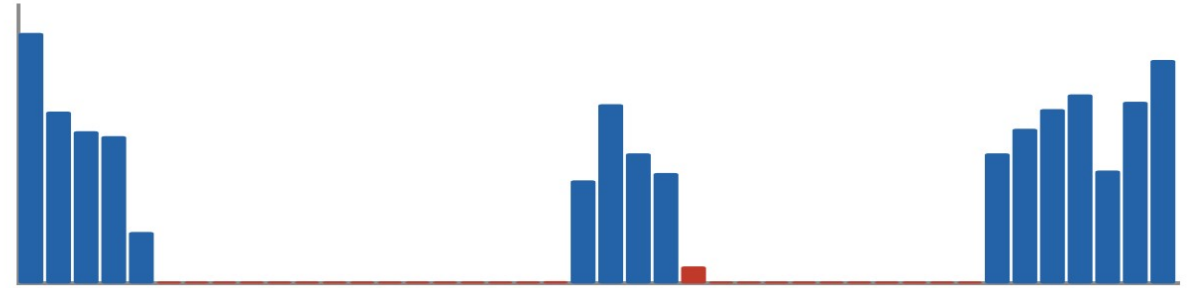
Immunology-defying claims like 'T cells differentiate into monocytes' enter the paper with a beautiful figure attached.

The cold open dissected: three tells of a fake trajectory

the cold-open "path"



discrete types: pseudotime density has canyons



genuine differentiation: continuous, no holes



pseudotime

one-line check: $\text{hist}(\text{pseudotime})$ — big empty stretches mean the path was forced

simulated data (algorithms run for real)

A branch point at a type boundary is an artefact; one line — $\text{hist}(\text{pseudotime})$ — shows the empty middle of a forced road

Trap 2: an arbitrary root

What it is

Filling in `start.clus` casually — or letting the first cluster in the default ordering win.



Why it happens

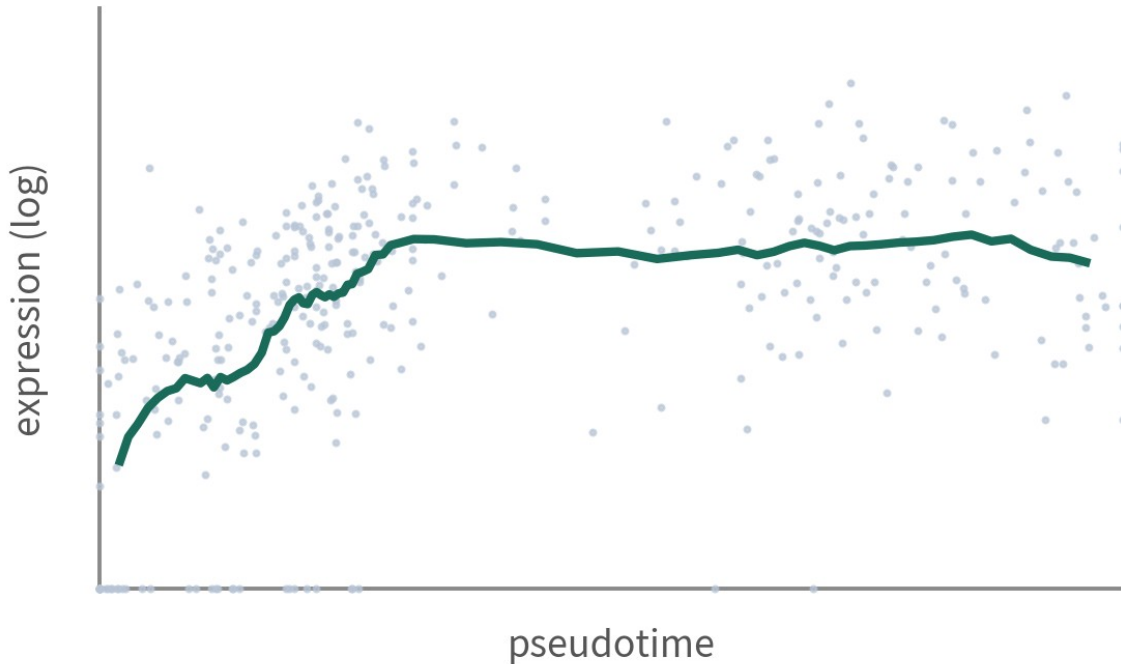
No error, equally pretty plots — a wrong direction raises no alarm at all.

What it costs you

The whole pseudotime reverses and every gene dynamic reads backwards: rises become falls, conclusions invert.

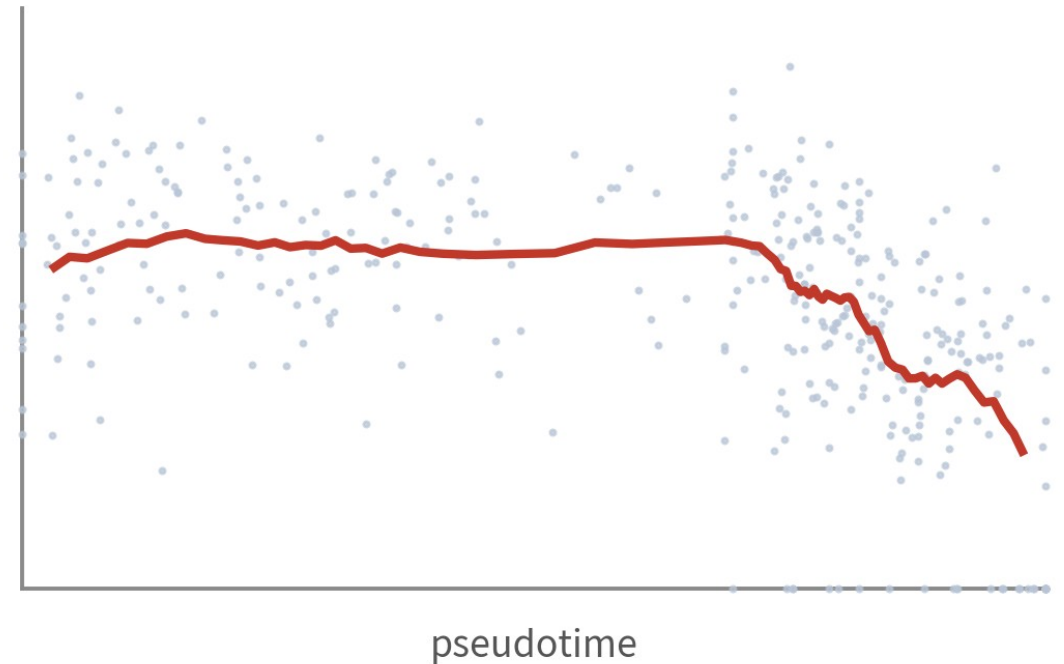
Wrong root, backwards biology

correct root



"maturation marker rises" — as biology says

arbitrary root



"cells LOSE the maturation marker" — backwards

same gene, same curve — swap the root and the whole biological story runs backwards

simulated data (algorithms run for real)

Same gene, same curve: swap the root and 'gaining the maturation marker' becomes 'losing' it

Trap 3: computing pseudotime on UMAP coordinates

What it is

Handing UMAP's 2-D coordinates to the trajectory tool, then treating the resulting pseudotime as quantitative.



Why it happens

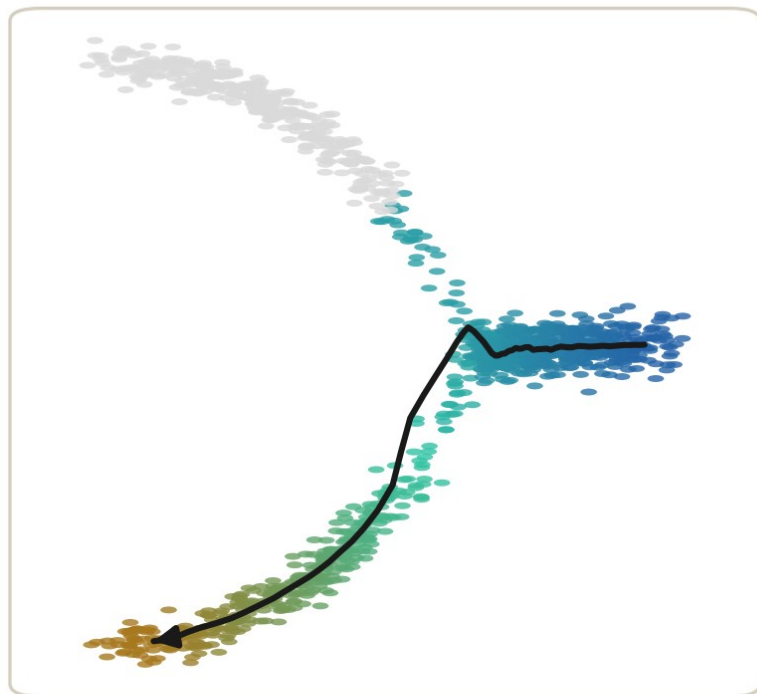
The UMAP is pretty and already there; 'coordinates are coordinates' is a tempting instinct.

What it costs you

UMAP compresses and stretches distances for layout (B9), so arc length distorts — local order scrambles, and the trunk may tear into islands.

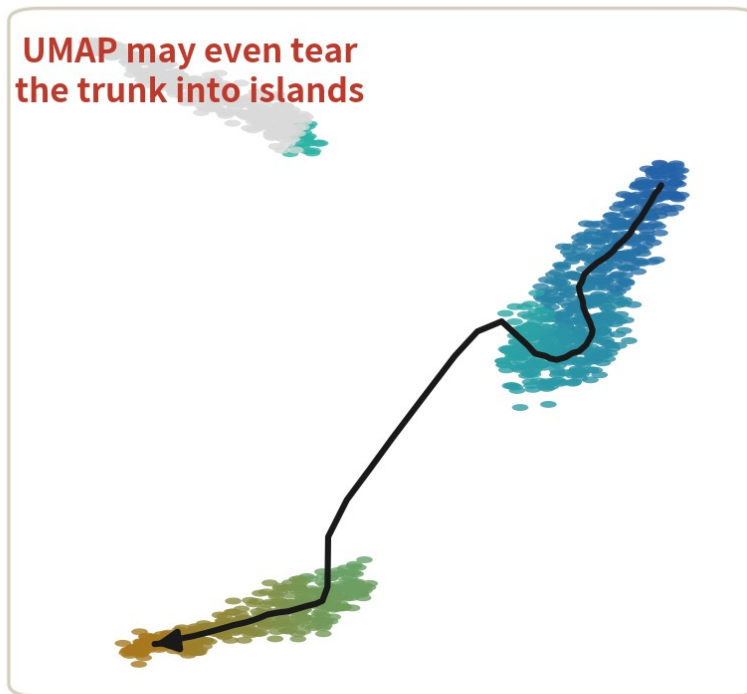
Same data: computed in PCA vs on UMAP

computed in PCA space (right)



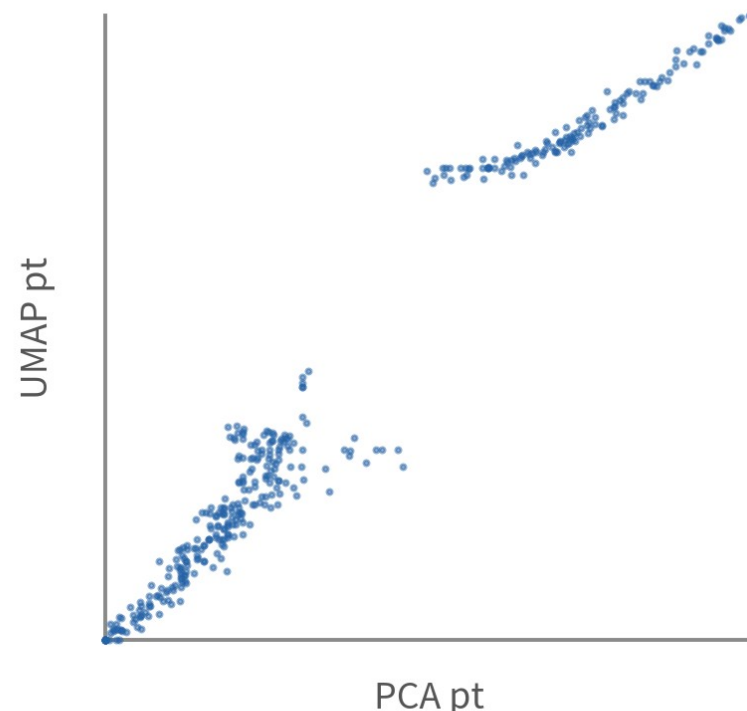
vs truth: $\rho = 0.99$

computed on UMAP coords (trap)



vs truth: $\rho = 0.98$

rank agreement $\rho = 0.98$



globally similar; local order
scrambled by UMAP's stretching

UMAP compresses and stretches distances to look good (B9) — its coordinates are a distorted map, not a ruler

Globally similar, locally scrambled — by how much depends on that particular UMAP's contortions. Not something to bet on

TRAPS, SUMMED UP

**Ask 'should I run this at all'
before asking 'how'.**

The checklist grants the licence, the root sets the direction, PCA space preserves distance —
only then does the curve deserve interpretation.

07+

Strategy Room: Complex Samples

The same trajectory tools, on a tumour — the first thing to re-ask is the premise itself

Premise check: does this tumour have a hierarchy?

Checklist question one, tumour edition — different tumours, opposite answers

GBM (solid tumour)

No hierarchy — four-state plasticity

- malignant cells shuttle among the four Neftel states (MES / AC / OPC / NPC-like): reversible, no fixed endpoint
- the askable question is state transition, not differentiation — and only within one patient's malignant cells
- independent evidence at both ends: state scores pick the root and verify the endpoint; cycle scores must not dominate

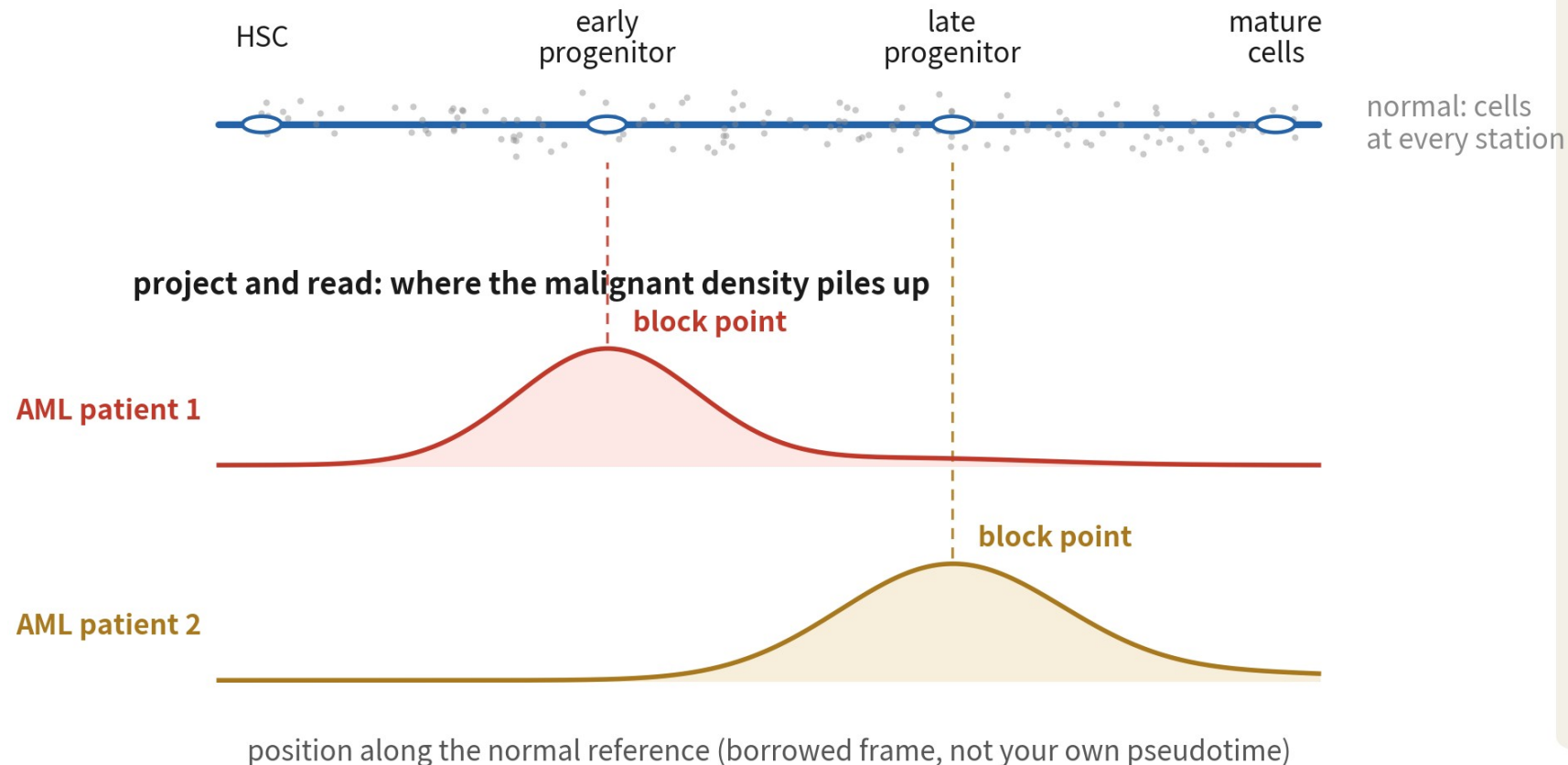
AML (blood cancer)

The hierarchy exists — malignancy stalls mid-way

- normal haematopoiesis is a textbook differentiation tree: every station from stem cell to mature blood has a name
- AML's defining feature is a differentiation block: malignant cells stop at a progenitor station and go no further
- the question is not 'how do they travel' but 'where are they stuck' — answered against a normal reference

The normal reference frame: normal marrow as coordinates

the borrowed frame: normal marrow's differentiation axis



what the frame buys you



borrowed axes

the normal path and stations are known; malignant cells just project



readable block

where density piles = where it stalls; per-patient projections compare



direction for free

start and end come from normal differentiation — no root guessing



same for GBM

normal brain lineages mirror the AC/OPC/NPC states; scores are axes

schematic

Malignant cells get no trajectory of their own — project them onto the normal reference and read where they stall or drift; for GBM, normal brain lineages play the same role

Trajectories do not compare across patients

Malignant cells island by patient and pseudotime has no shared unit — comparison needs a different statistic

Pooled = riding the CNV



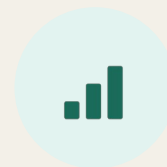
pool patients' malignant cells and the MST bridges the patient islands first — that 'road' traces genomic differences, not state transitions

One patient, one trajectory



recompute the whole pipeline after subsetting; pick the root by top state score, never a hard-coded cluster id — the same code runs on the next patient

Compare states, not arc length



pseudotime is mileage on each patient's own curve, with no shared unit; compare state scores and proportions — the patient is the statistical unit

Longitudinal trajectories around treatment: three traps

The most tempting design — every row has claimed victims

Trap	Why it fails	What to do instead
Pool pre and post into one trajectory: 'treatment pushed cells forward'	pre and post are two independent snapshots, treatment fully confounded with batch; the tool paves a road regardless	analyse each time point separately; compare states and proportions on a shared reference
Compare raw pseudotime values between samples	arc length depends on each curve and its cell density — two trajectories share no unit	project onto one shared reference, then compare distributions (Appendix F, Q8)
Reading a composition shift as a state transition	treatment kills the sensitive cells and the survivors' mix shifts — selection is not transition, and snapshots cannot tell them apart	needs clone-level evidence (B16's CNV subclones) or true multi-time-point tracking; otherwise report both readings

Episode checklist

- | | |
|---|--|
| ✓ before running: continuous? intermediates? root evidence? | ✓ trajectory computed in PCA space; UMAP is display only |
| ✓ root chosen from biology (markers / time points) | ✓ re-ran with another root — direction is no accident |
| ✓ checked hist(pseudotime): continuous density, no canyons | ✓ dynamics from smoothers / tradeSeq, not raw dots |

All six boxes ticked — then your trajectory has a root, a direction, and answers

ONE-LINE SUMMARY

**A trajectory answers 'how cells travel' —
but 'whether they travel' you must answer first.**

Eligibility (checklist) → skeleton (MST) → road (curves) → time (pseudotime) → dynamics
(tradeSeq).

Check yourself 1

What is pseudotime, fundamentally?

- A Wall-clock time since the cells were sampled
- B An ordering of cells along the curve (arc length) — not physical time
- C The number of divisions a cell has undergone
- D The first UMAP coordinate

Answer B. The snapshot paradox: no cell was tracked — cells caught at different progress were ordered by similarity. Pseudotime is arc length along the curve; its units are neither hours nor days. An ordering, not a clock.

Check yourself 2

What do `getLineages` and `getCurves` each do?

- A The former computes pseudotime; the latter plots
- B The former runs UMAP; the latter runs PCA
- C The former builds the MST over cluster centroids (the skeleton); the latter fits smooth principal curves
- D They are equivalent; use either

Answer C. `getLineages` builds a minimum spanning tree over cluster centroids — which clusters, where it forks; `getCurves` then iteratively fits a principal curve per lineage, and pseudotime is arc length along it. Skeleton first, road second.

Check yourself 3

**You get tumour-infiltrating immune cells and the boss says 'run a trajectory'.
Step one?**

- A Just run Slingshot — it will produce something
- B Run the three-question checklist first: continuous? intermediates? root evidence?
- C Run RNA velocity first because it is newer
- D Feed the UMAP coordinates to Monocle3

Answer B. The tool always returns a curve, so eligibility must be judged before running. A T-cell exhaustion continuum may qualify; a handful of discrete mature types should stop you. And after running, check `hist(pseudotime)` for canyons.

NEXT EPISODE

B16: inferCNV and calling malignant cells

Trajectories assume cells change continuously — and normally. But in a tumour, some cells differ at the genome itself.

Three epithelial clusters, markers everywhere: which one is cancer? Clustering cannot tell you. Next episode we catch them on the chromosomes.